COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	07/22/2026	**DATE SUBMITTED:**	
NAME:	Canoy Hail B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 3.1 **Hands on Activity 3.1 Linked List**

1. PROCEDURE OUTPUT

Output

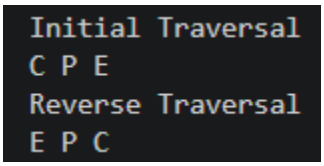
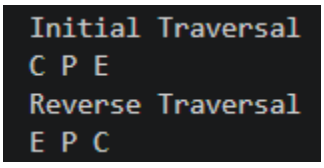
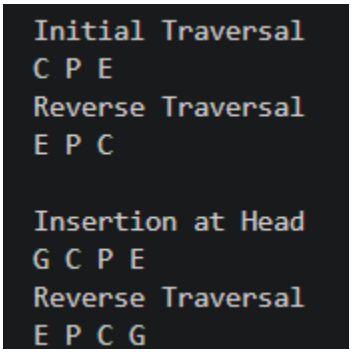
Screenshot	
Discussion	The program successfully created the double linked list. With the letter CPE as nodes. We did it using pointers next and prev links.

Table 3-1. Output of initial Simple Implementation

Operation	Screenshot
Traversal	
Insertion at head	

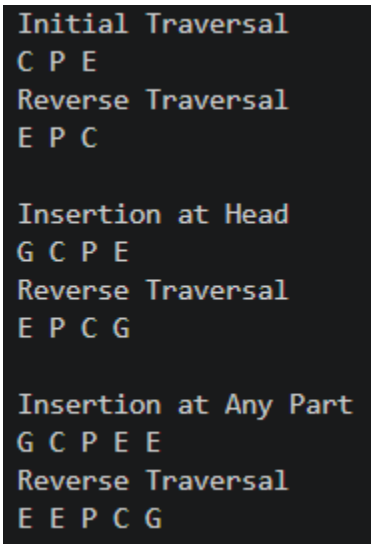
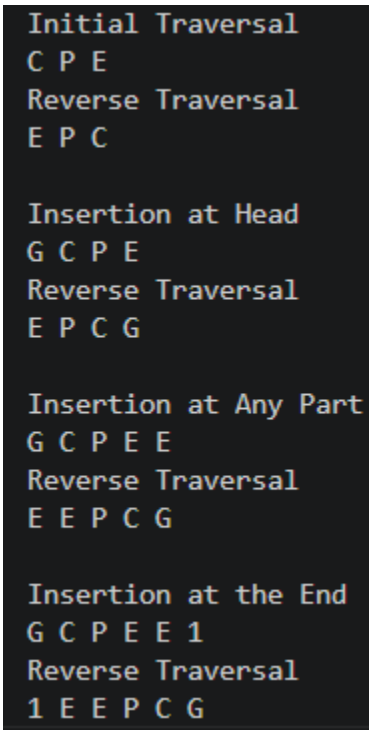
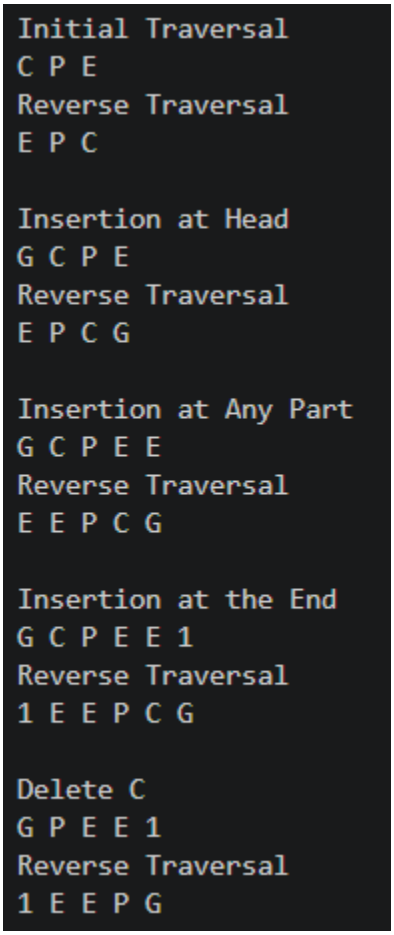
Insertion at any part of the list	
Insertion at the end	

Table 3-2. Code for the List Operations

a.	Source Code	Main.cpp, double_LL.h, singly_LL.h
	Console	Initial Traversal C P E Reverse Traversal E P C
b.	Source Code	Main.cpp with dllInsertHeaf, double_LL.h, singly_LL.h
	Console	Initial Traversal C P E Reverse Traversal E P C Insertion at Head G C P E Reverse Traversal E P C G
c.	Source Code	Main.cpp with dllInsertMiddle, double_LL.h with dllInsertMiddle, singly_LL.h
	Console	Initial Traversal C P E Reverse Traversal E P C Insertion at Head G C P E Reverse Traversal E P C G Insertion at Any Part G C P E E Reverse Traversal E E P C G

d.	Source Code	Main.cpp with dllInsertEnd, double_LL.h, singly_LL.h
	Console	Initial Traversal C P E Reverse Traversal E P C Insertion at Head G C P E Reverse Traversal E P C G Insertion at Any Part G C P E E Reverse Traversal E E P C G Insertion at the End G C P E E 1 Reverse Traversal 1 E E P C G
e.	Source Code	Main.cpp with updated dllDelete c and &head, double_LL.h added with dllDelete, singly_LL.h

	Console	
f.	Source Code	Main.cpp with updated dllDelete p and &head, double_LL.h, singly_LL.h

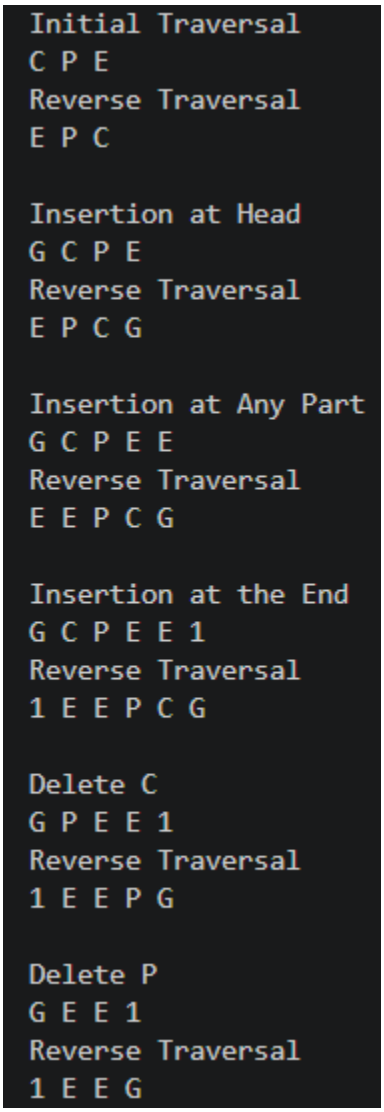
	Console	

Table 3-3. Code and Analysis for the Singly Linked List

Screenshot	Analysis




```cpp
#ifndef DOUBLE_LL_H
#define DOUBLE_LL_H

#include <iostream>

template <typename T>
class doubleList {
public:
    T data;
    doubleList<T>* next = nullptr;
    doubleList<T>* prev = nullptr;
};

//============================
// Create New Node
//============================
template <typename T>
doubleList<T>* new_node(T newData)
{
    doubleList<T>* newNode = new doubleList<T>;

    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}

//============================
// Traversal
//============================
template <typename T>
void dllTraverse(doubleList<T>* currentNode)
{
    doubleList<T>* tail = nullptr;

    while (currentNode != nullptr)
    {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    std::cout << std::endl;

    std::cout << "Reverse Traversal" << std::endl;

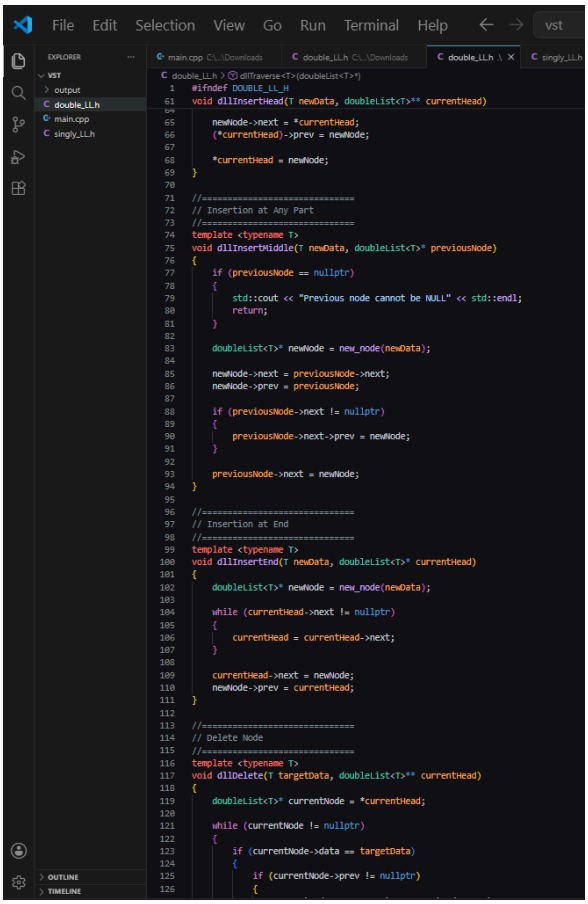
    while (tail != nullptr)
    {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }

    std::cout << std::endl;
}

//============================
// Insertion at Head
//============================
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead)
{
    doubleList<T>* newNode = new_node(newData);

    newNode->next = *currentHead;
```

- The program is divided into three files to organize the implementation of the doubly linked lists. We have the files named double_LL.h, the singly__LL.h and of course the main cpp file names main.cpp. The main.cpp serves as the driver program that creates the linked list and calls the different operations, The double_LL.h file contains the definition of the doubly linked list node and the functions for the node creation and things such us deletion, insertion of the node, position, traversal. The singly_LL.h does not contain any implementation. We separated the program into different files to make the code more organized and easier to read and reuse.



```
#ifndef DOUBLE_LL_H
void dllInsertHead(T newData, doubleList<T>** currentHead)

    newNode->next = *currentHead;
    (*currentHead)->prev = newNode;

    *currentHead = newNode;
}

//============================
// Insertion at Any Part
//============================
template <typename T>
void dllInsertMiddle(T newData, doubleList<T>* previousNode)
{
    if (previousNode == nullptr)
    {
        std::cout << "Previous node cannot be NULL" << std::endl;
        return;
    }

    doubleList<T>* newNode = new_node(newData);

    newNode->next = previousNode->next;
    newNode->prev = previousNode;

    if (previousNode->next != nullptr)
    {
        previousNode->next->prev = newNode;
    }

    previousNode->next = newNode;
}

//============================
// Insertion at End
//============================
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead)
{
    doubleList<T>* newNode = new_node(newData);

    while (currentHead->next != nullptr)
    {
        currentHead = currentHead->next;
    }

    currentHead->next = newNode;
    newNode->prev = currentHead;
}

//============================
// Delete Node
//============================
template <typename T>
void dllDelete(T targetData, doubleList<T>** currentHead)
{
    doubleList<T>* currentNode = *currentHead;

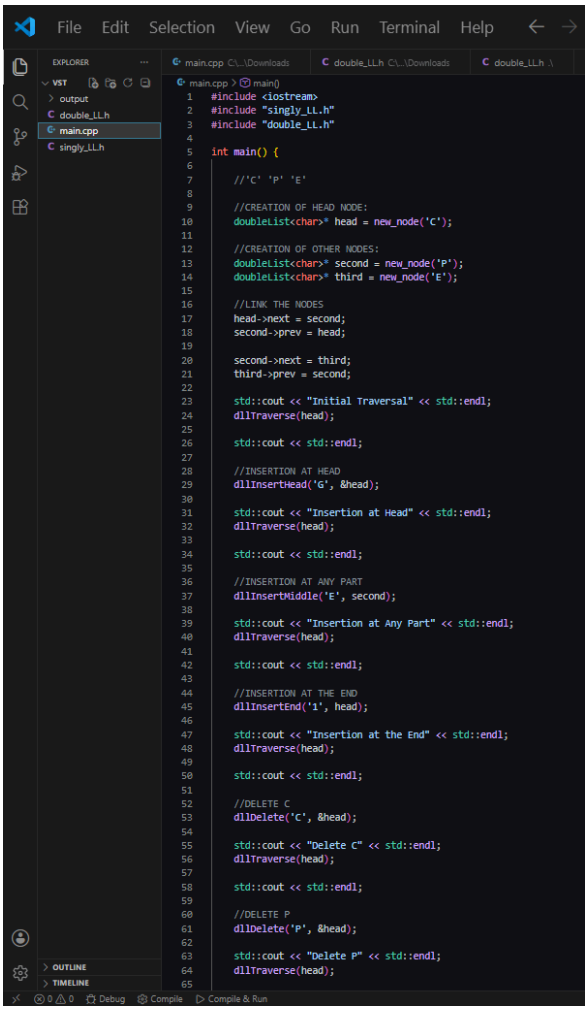
    while (currentNode != nullptr)
    {
        if (currentNode->data == targetData)
        {
            if (currentNode->prev != nullptr)
            {
                currentNode->prev->next = currentNode->next;
            }
            else
            {
                *currentHead = currentNode->next;
            }

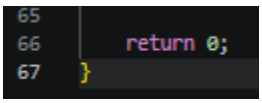
            if (currentNode->next != nullptr)
            {
                currentNode->next->prev = currentNode->prev;
            }

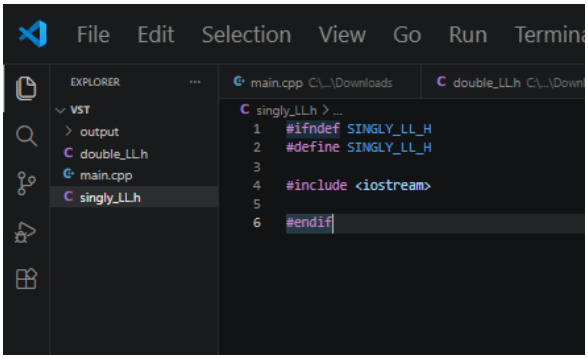
            delete currentNode;
            break;
        }

        currentNode = currentNode->next;
    }
}

#endif
```


File Edit Selection View Go Run Terminal Help
EXPLORER
VST
output
double_LL.h
main.cpp
singly_LL.h
main.cpp C:\...\Downloads
double_LL.h C:\...\Downloads
main.cpp > main()
#include <iostream>
#include "singly_LL.h"
#include "double_LL.h"
int main() {
//'C' 'P' 'E'
//CREATION OF HEAD NODE:
doubleList<char>* head = new_node('C');
//CREATION OF OTHER NODES:
doubleList<char>* second = new_node('P');
doubleList<char>* third = new_node('E');
//LINK THE NODES
head->next = second;
second->prev = head;
second->next = third;
third->prev = second;
std::cout << "Initial Traversal" << std::endl;
dllTraverse(head);
std::cout << std::endl;
//INSERTION AT HEAD
dllInsertHead('G', &head);
std::cout << "Insertion at Head" << std::endl;
dllTraverse(head);
std::cout << std::endl;
//INSERTION AT ANY PART
dllInsertMiddle('E', second);
std::cout << "Insertion at Any Part" << std::endl;
dllTraverse(head);
std::cout << std::endl;
//INSERTION AT THE END
dllInsertEnd('1', head);
std::cout << "Insertion at the End" << std::endl;
dllTraverse(head);
std::cout << std::endl;
//DELETE C
dllDelete('C', &head);
std::cout << "Delete C" << std::endl;
dllTraverse(head);
std::cout << std::endl;
//DELETE P
dllDelete('P', &head);
std::cout << "Delete P" << std::endl;
dllTraverse(head);
OUTLINE
TIMELINE
Debug Compile Compile & Run


return 0;
}


File Edit Selection View Go Run
EXPLORER
VST
output
double_LL.h
main.cpp
singly_LL.h
main.cpp C:\...\Downloads
singly_LL.h
#ifndef SINGLY_LL_H
#define SINGLY_LL_H
#include <iostream>
#endif

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

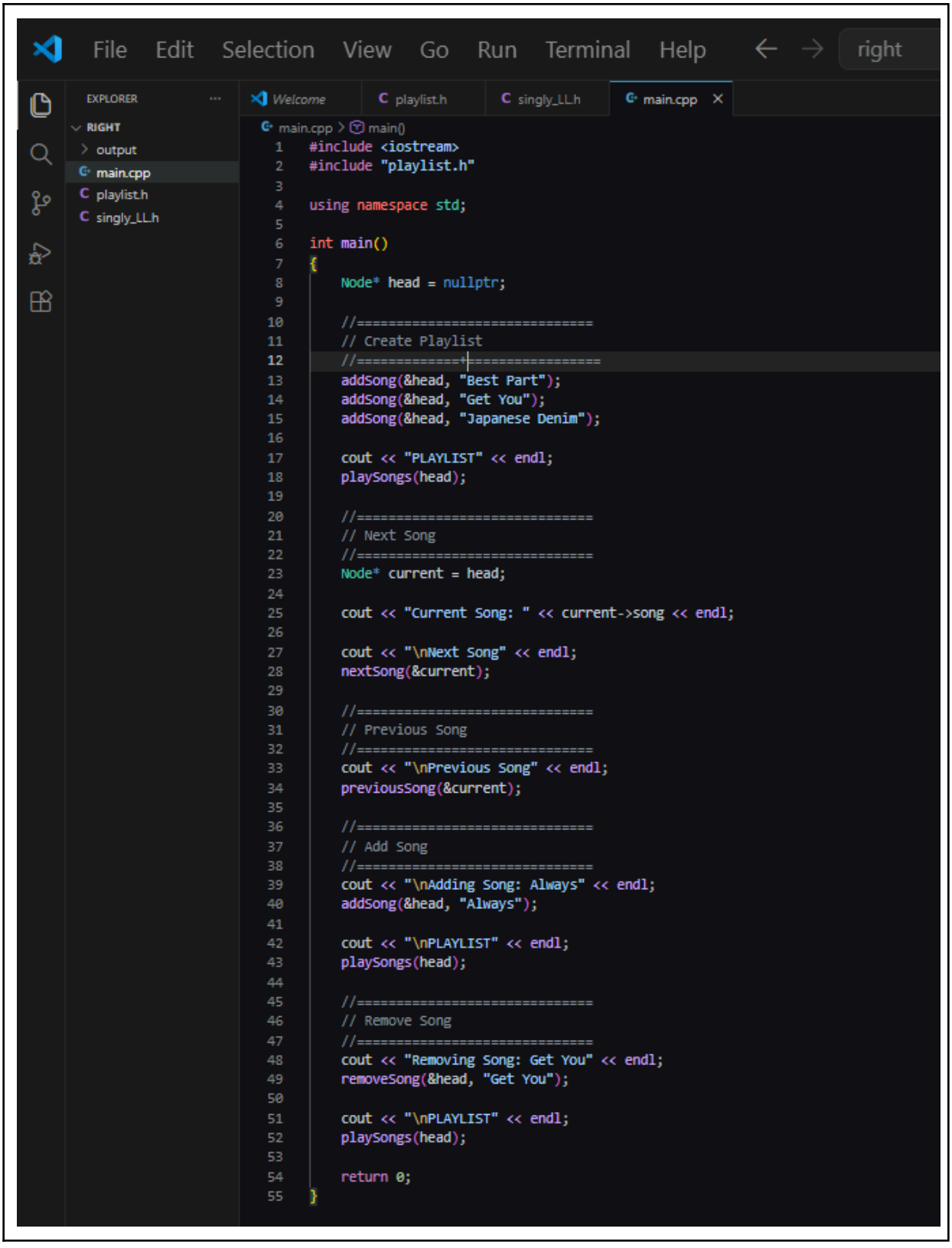
- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

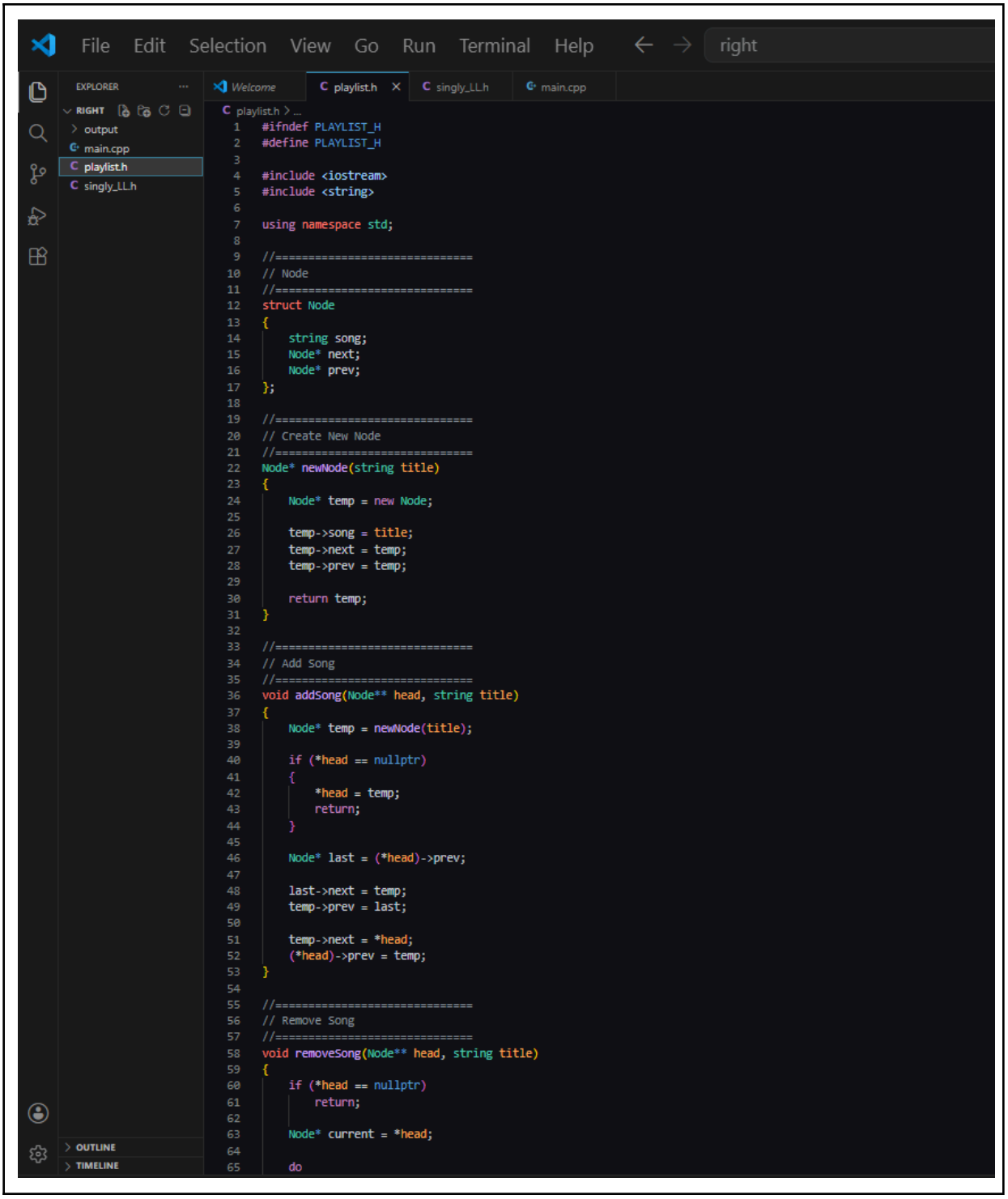
Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

Code:


File Edit Selection View Go Run Terminal Help
right
EXPLORER
RIGHT
output
main.cpp
playlist.h
singly_LL.h
Welcome
playlist.h
singly_LL.h
main.cpp
main.cpp > main()
#include <iostream>
#include "playlist.h"
using namespace std;
int main()
{
Node* head = nullptr;
//==============================
// Create Playlist
//==============================
addSong(&head, "Best Part");
addSong(&head, "Get You");
addSong(&head, "Japanese Denim");
cout << "PLAYLIST" << endl;
playSongs(head);
//==============================
// Next Song
//==============================
Node* current = head;
cout << "Current Song: " << current->song << endl;
cout << "\nNext Song" << endl;
nextSong(&current);
//==============================
// Previous Song
//==============================
cout << "\nPrevious Song" << endl;
previousSong(&current);
//==============================
// Add Song
//==============================
cout << "\nAdding Song: Always" << endl;
addSong(&head, "Always");
cout << "\nPLAYLIST" << endl;
playSongs(head);
//==============================
// Remove Song
//==============================
cout << "Removing Song: Get You" << endl;
removeSong(&head, "Get You");
cout << "\nPLAYLIST" << endl;
playSongs(head);
return 0;
}




```cpp
#ifndef PLAYLIST_H
#define PLAYLIST_H

#include <iostream>
#include <string>

using namespace std;

//============================
// Node
//============================
struct Node
{
    string song;
    Node* next;
    Node* prev;
};

//============================
// Create New Node
//============================
Node* newNode(string title)
{
    Node* temp = new Node;

    temp->song = title;
    temp->next = temp;
    temp->prev = temp;

    return temp;
}

//============================
// Add Song
//============================
void addSong(Node** head, string title)
{
    Node* temp = newNode(title);

    if (*head == nullptr)
    {
        *head = temp;
        return;
    }

    Node* last = (*head)->prev;

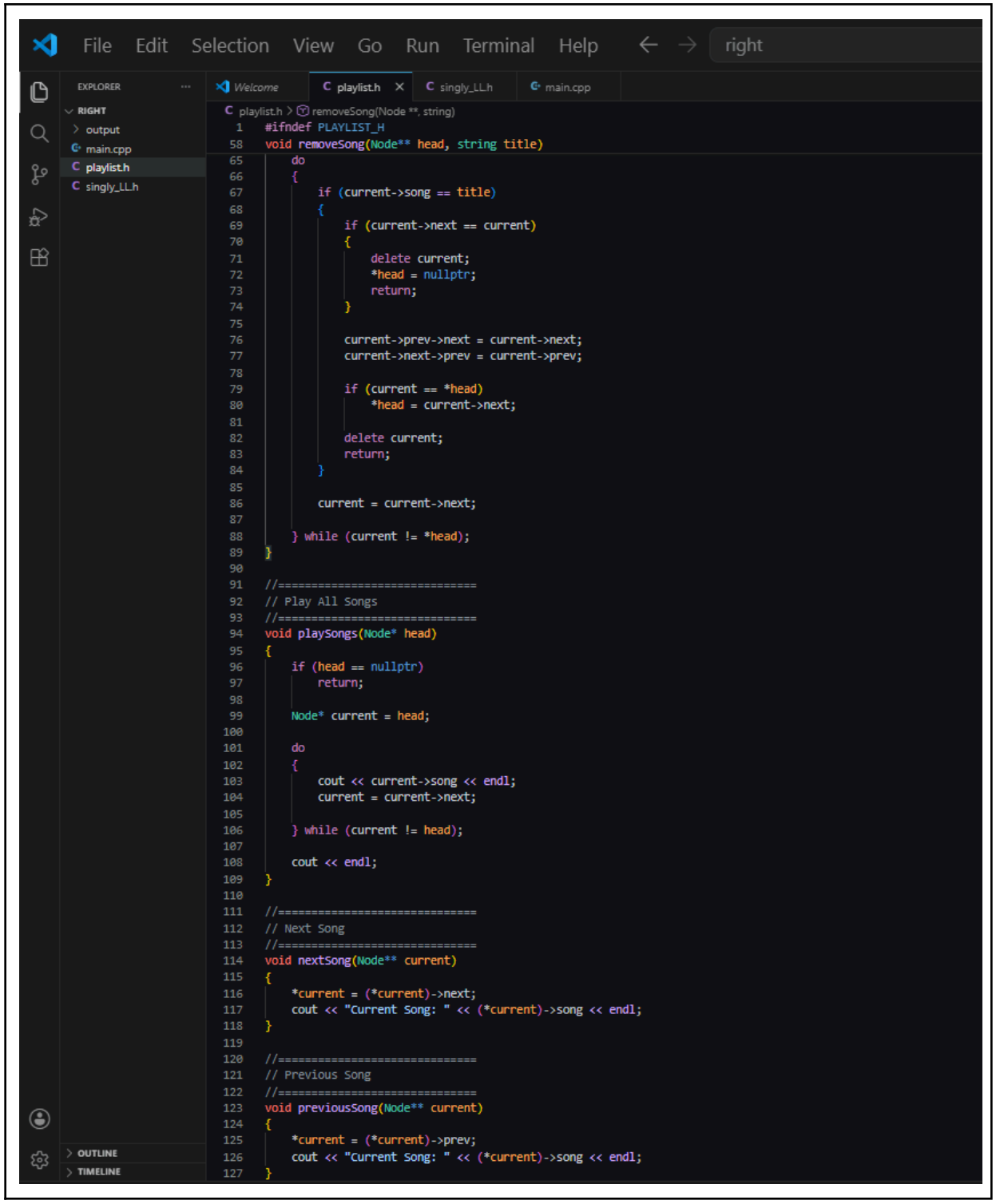
    last->next = temp;
    temp->prev = last;

    temp->next = *head;
    (*head)->prev = temp;
}

//============================
// Remove Song
//============================
void removeSong(Node** head, string title)
{
    if (*head == nullptr)
        return;

    Node* current = *head;

    do
```




```cpp
#ifndef PLAYLIST_H
void removeSong(Node** head, string title)
    do
    {
        if (current->song == title)
        {
            if (current->next == current)
            {
                delete current;
                *head = nullptr;
                return;
            }

            current->prev->next = current->next;
            current->next->prev = current->prev;

            if (current == *head)
                *head = current->next;

            delete current;
            return;
        }

        current = current->next;

    } while (current != *head);
}

//============================
// Play All Songs
//============================
void playSongs(Node* head)
{
    if (head == nullptr)
        return;

    Node* current = head;

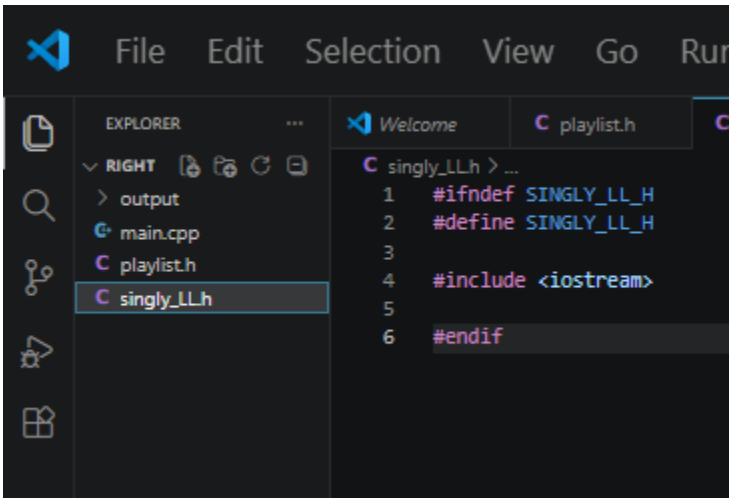
    do
    {
        cout << current->song << endl;
        current = current->next;

    } while (current != head);

    cout << endl;
}

//============================
// Next Song
//============================
void nextSong(Node** current)
{
    *current = (*current)->next;
    cout << "Current Song: " << (*current)->song << endl;
}

//============================
// Previous Song
//============================
void previousSong(Node** current)
{
    *current = (*current)->prev;
    cout << "Current Song: " << (*current)->song << endl;
}
```



Output

```
PLAYLIST
Best Part
Get You
Japanese Denim

Current Song: Best Part

Next Song
Current Song: Get You

Previous Song
Current Song: Best Part

Adding Song: Always

PLAYLIST
Best Part
Get You
Japanese Denim
Always

Removing Song: Get You

PLAYLIST
Best Part
Japanese Denim
Always
```

Analysis

- My program also has 3 files names playlist.h, main.cpp and singly_LL.h. First the playlist.h file contains

the linked list structure and the functions for creating playlist, adding and removing songs and playing all songs then next is the main.cpp. The main.cpp file serves as a storage that demonstrates each required operation with the songs I love by the artist named Daniel Caesar. The last one is singly_LL.h doesn't have that much code and implementation but the program can run without it.

Conclusion:

- In this activity, I learned how to implement a circular linked list by creating a music playlist of Daniel Caesar. I learned how to create a playlist using it, add and remove songs. I also learned how a circular linked list allows the last node to connect back to the first node making it possible and to continuously cycle through the songs. This activity was really hard for me, although I understand the concept of it its still really complicated. Overall even though this activity was hard I still understand the basic concepts, property and how to implement the code properly, even though it's still hard and complicated at least I understand the basics of it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **I used it to help me understand the complex program in this activity.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I only used ChatGPT as a learning aid to explain and suggest how to improve my work while I reviewed and tested it myself.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.